```python
import math

# Board cells: empty= ' ', player 'X', opponent 'O'
board = [' ' for _ in range(9)]

def print_board():
    for i in range(3):
        print(board[3*i], '|', board[3*i+1], '|', board[3*i+2])
        if i < 2:
            print('---------')

def is_winner(b, player):
    wins = [
        (0,1,2), (3,4,5), (6,7,8), # rows
        (0,3,6), (1,4,7), (2,5,8), # columns
        (0,4,8), (2,4,6)           # diagonals
    ]
    for a, b_, c in wins:
        if b[a] == player and b[b_] == player and b[c] == player:
            return True
    return False

def is_board_full(b):
    return ' ' not in b

def minimax_alpha_beta(b, depth, alpha, beta, is_maximizing):
    if is_winner(b, 'X'):
        return 10 - depth
    if is_winner(b, 'O'):
        return depth - 10
    if is_board_full(b):
        return 0

    if is_maximizing:
        max_eval = -math.inf
        for i in range(9):
            if b[i] == ' ':
                b[i] = 'X'
                eval = minimax_alpha_beta(b, depth+1, alpha, beta, False)
                b[i] = ' '
                max_eval = max(max_eval, eval)
                alpha = max(alpha, eval)
                if beta <= alpha:
                    break
        return max_eval
    else:
        min_eval = math.inf
        for i in range(9):
            if b[i] == ' ':
                b[i] = 'O'
                eval = minimax_alpha_beta(b, depth+1, alpha, beta, True)
                b[i] = ' '
                min_eval = min(min_eval, eval)
                beta = min(beta, eval)
                if beta <= alpha:
                    break
        return min_eval

def best_move():
    best_score = -math.inf
    move = -1
    for i in range(9):
        if board[i] == ' ':
            board[i] = 'X'
            score = minimax_alpha_beta(board, 0, -math.inf, math.inf, False)
            board[i] = ' '
            if score > best_score:
                best_score = score
                move = i
    return move

def play_game():
    # Reset the board for a new game
    global board
    board = [' ' for _ in range(9)]
```

```
    print("Board positions are numbered 0 to 8:")
    print("0 | 1 | 2")
    print("---------")
    print("3 | 4 | 5")
    print("---------")
    print("6 | 7 | 8")
    print()

    while True:
        move = best_move()
        if move == -1:
            print("Game Over! It's a tie.")
            break
        board[move] = 'X'
        print("AI (X) plays move:", move)
        print_board()
        if is_winner(board, 'X'):
            print("AI (X) wins!")
            break
        if is_board_full(board):
            print("Game Over! It's a tie.")
            break

        while True:
            try:
                human_move = int(input("Enter your move (0-8): "))
                if 0 <= human_move <= 8 and board[human_move] == ' ':
                    board[human_move] = 'O'
                    break
                else:
                    print("Invalid move. Try again.")
            except ValueError:
                print("Please enter a valid number.")

        print_board()
        if is_winner(board, 'O'):
            print("You Win!")
            break
        if is_board_full(board):
            print("Game Over! It's a tie.")
            break

# To play, call play_game()
play_game()
```

```
Board positions are numbered 0 to 8:
0 | 1 | 2
---------
3 | 4 | 5
---------
6 | 7 | 8

AI (X) plays move: 0
X |   |
---------
  |   |
---------
  |   |
Enter your move (0-8): 6
X |   |
---------
  |   |
---------
O |   |
AI (X) plays move: 1
X | X |
---------
  |   |
---------
O |   |
Enter your move (0-8): 2
X | X | O
---------
  |   |
---------
O |   |
AI (X) plays move: 4
X | X | O
---------
```

```
  | X |
---------
O |   |
Enter your move (0-8): 7
X | X | O
---------
  | X |
---------
O | O |
AI (X) plays move: 8
X | X | O
---------
  | X |
---------
O | O | X
AI (X) wins!
```